

Rapport Détaillé des Fonctions du Projet Farm AI

Ce document présente une vue d'ensemble de l'architecture du projet **Farm AI** (Système de Surveillance Intelligente) et détaille les fonctions clés implémentées dans les trois services principaux : l'Intelligence Artificielle (Python/FastAPI), le Backend (Java/Spring Boot) et le Frontend (Angular).

1. Module Intelligence Artificielle (Python / FastAPI)

Ce module est responsable de la reconnaissance faciale, de l'analyse vidéo en temps réel et de la détection de l'équipement vestimentaire (couleurs, uniformes) grâce aux modèles InsightFace et YOLOv8.

Fonctions et Endpoints Principaux :

main.py (API FastAPI)

- **register_face(employeeId: int)** : Active la caméra en temps réel pour capturer le visage d'un employé. Extrait les *embeddings* (vecteurs caractéristiques du visage) via le modèle `buffalo_l` (InsightFace) et les enregistre dans la base de données (table `employees`).
- **recognize_face()** : Ouvre le flux vidéo pour identifier une personne. Compare les embeddings détectés en direct avec ceux stockés dans la base de données. Retourne l'ID de l'employé avec un score de confiance (*confidence*) si la distance vectorielle est inférieure au seuil autorisé.
- **delete_face(employeeId: int)** : Supprime les données biométriques (embeddings) associées à un employé spécifique pour des raisons de confidentialité ou de réinitialisation.

face_live.py & clothes_live.py (Scripts de Surveillance)

- **process_frame(frame)** : Fonction cœur qui prend une image (frame) de la caméra, lance la détection de visage (InsightFace) ou la détection d'objets (YOLOv8) pour repérer les personnes.
- **detect_clothing_color(crop_img)** : Analyse une zone recadrée de l'image (le corps de la personne) pour déterminer la couleur dominante des vêtements (ex: rouge, bleu, etc.) en convertissant l'image en espace colorimétrique HSV.
- **send_alert(person_data)** : Envoie une requête HTTP POST au Backend Spring Boot dès qu'une personne non autorisée ou un manquement à l'uniforme est détecté.

2. Module Backend (Java / Spring Boot)

Le cœur logique de l'application. Il gère la sécurité (JWT), l'accès aux données (MySQL via Hibernate/JPA), et orchestre les communications entre l'interface utilisateur et le système d'IA.

Contrôleurs (Controllers) et Services Clés :

AuthController & AuthService

- **login(LoginRequest request)** : Authentification classique par email et mot de passe. Vérifie les credentials, génère un token JWT via `JwtUtil` et retourne le rôle de l'utilisateur (Admin, Manager, Viewer).
- **faceLogin()** : Fait appel au service FastAPI Python pour déclencher la reconnaissance faciale, et connecte l'utilisateur si le visage correspond à un compte actif.
- **resetPassword(email)** : Génère un token de réinitialisation temporaire et envoie un email via `EmailService`.

EmployeeController & EmployeeService

- **getAllEmployees() / getEmployeeById(id)** : Récupère la liste ou le détail des employés depuis la base de données.
- **addEmployee(EmployeeDTO dto)** : Crée un nouvel employé, lui associe un département, génère automatiquement un compte `User` lié et déclenche l'envoi d'un email de bienvenue avec le mot de passe initial.
- **updateEmployeeStatus()** : Met à jour l'état de l'employé (Présent, Absent, En pause) en fonction des détections de l'IA.

CameraController & AlertController

- **registerCamera(CameraDTO dto)** : Ajoute une nouvelle caméra de surveillance au système et l'affecte à un département.
- **receiveIAAlert(AlertRequest alert)** : Endpoint sécurisé (API interne) appelé par les scripts Python pour signaler une intrusion ou une détection anormale.
- **getRecentAlerts()** : Récupère les 50 dernières alertes pour les afficher en temps réel sur le tableau de bord Frontend.

AttendanceController

- **recordAttendance(AttendanceDTO dto)** : Enregistre l'heure d'arrivée (Check-in) ou de départ (Check-out) d'un employé, calculée automatiquement suite à une détection par l'IA à l'entrée de la ferme.

3. Module Frontend (Angular)

L'interface utilisateur permet aux administrateurs et managers de visualiser les données, gérer les employés et observer les caméras en direct.

Composants et Méthodes (Components) :

LoginComponent (login.component.ts)

- **login()** : Valide le formulaire, appelle l'API d'authentification du Backend, stocke le JWT dans le `localStorage` et redirige selon le rôle.
- **faceLogin()** : Déclenche l'animation de chargement et appelle l'API de reconnaissance faciale. En cas de succès, connecte l'utilisateur sans mot de passe.

DashboardComponent (dashboard.component.ts)

- **loadStatistics()** : Récupère les métriques clés (nombre d'employés, alertes du jour, présences) via `HttpClient` pour mettre à jour les cartes en haut du tableau de bord.
- **initCharts()** : Utilise `Chart.js` pour dessiner les graphiques de fréquentation hebdomadaire et de performance.
- **pollLiveAlerts()** : Exécute une requête `setInterval` toutes les 5 secondes pour récupérer les dernières alertes générées par l'IA et les afficher dans le fil d'actualité.

FaceSetupComponent (face-setup.component.ts)

- **startRegistration()** : Guide le nouvel utilisateur lors de sa première connexion pour enregistrer son visage dans le système. Appelle l'endpoint `/api/face/register` du Backend (qui lui-même appelle l'IA).

Services Angular (auth.service.ts, api.service.ts)

- **AuthGuard (canActivate)** : Intercepteur de route qui empêche l'accès aux pages internes si le token JWT n'est pas présent ou expiré.
- **AuthInterceptor** : Intercepte toutes les requêtes HTTP sortantes pour y attacher automatiquement l'en-tête `Authorization: Bearer <token>`.

Ce rapport a été généré pour documenter la structure du projet Fam AI.